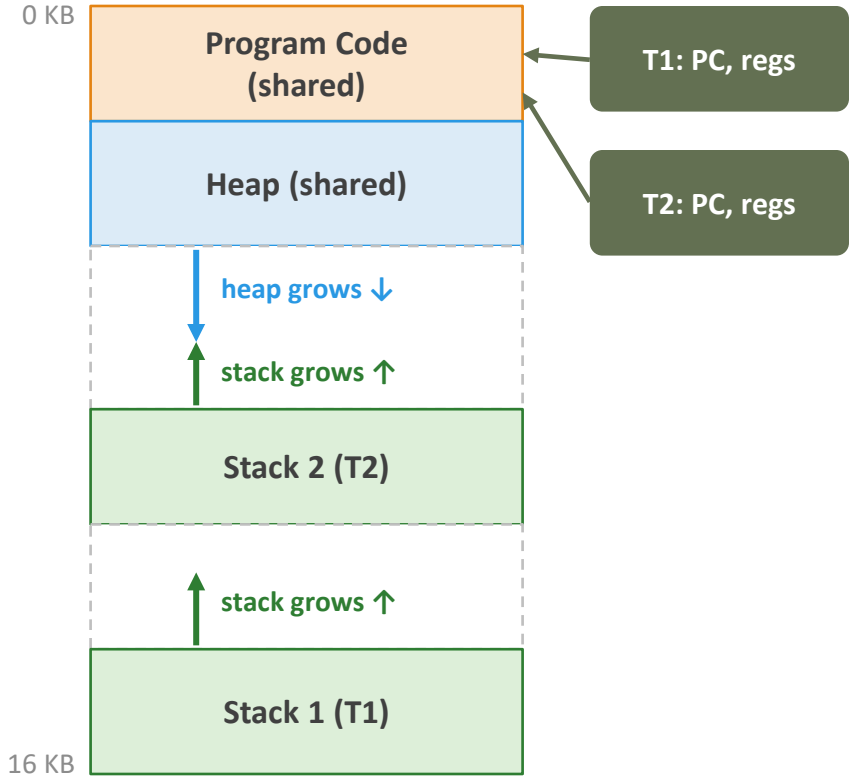
CS3.301: Operating Systems and Networks
Concurrency: Semaphores

INSTRUCTOR: DR. DEEPAK GANGADHARAN

Recap: Threads Share One Address Space

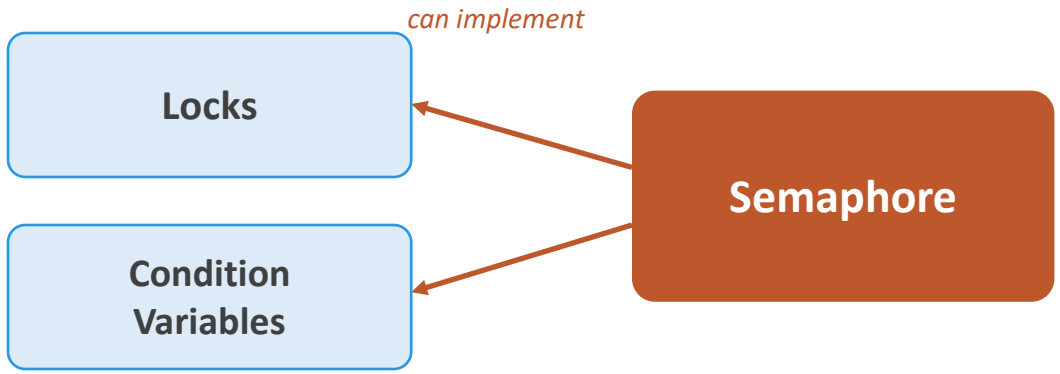
- Threads of a process share **code**, **globals** and the **heap**
- Each thread has its own **PC**, **registers** and **stack**
  - Locals on a thread's stack are private (thread-local)
- Shared data + arbitrary interleaving → **race conditions**
- Code that accesses shared data = **critical section**
- Tools so far:
  - **Locks** → mutual exclusion
  - **Condition variables** → wait for a condition (ordering)
- **Today: one primitive that does both — the semaphore**



Address space of a process with two threads

Semaphores: One Primitive for Locks and CVs

- Introduced by **Edsger Dijkstra** (THE system, 1960s)
  - Also: shortest paths, "Goto Considered Harmful"
- A **single** synchronization primitive
- Can be used as **both** a lock and a condition variable



The Crux: How to Use Semaphores

1. How do we use semaphores instead of locks and condition variables?
2. What is the definition of a semaphore?
3. What is a binary semaphore?
4. Can we build a semaphore out of locks and CVs?
5. Can we build locks and CVs out of semaphores?

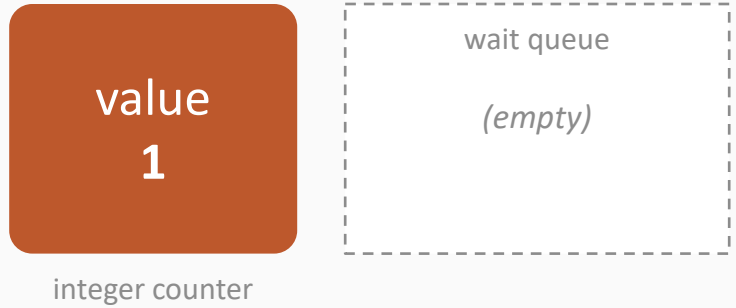
A Semaphore: Definition and Initialization

- An object with an **integer value** + two routines
  - POSIX: `sem_wait()` and `sem_post()`
  - Dijkstra: **P()** and **V()** (Dutch words); also down/up
- Initial value determines behaviour → **must initialize first**

```
#include <semaphore.h>
sem_t s;
sem_init(&s, 0, 1);
```

arg 2 = **0** → shared by threads of the **same process**
arg 3 = **1** → the **initial value**

Mental model of semaphore s



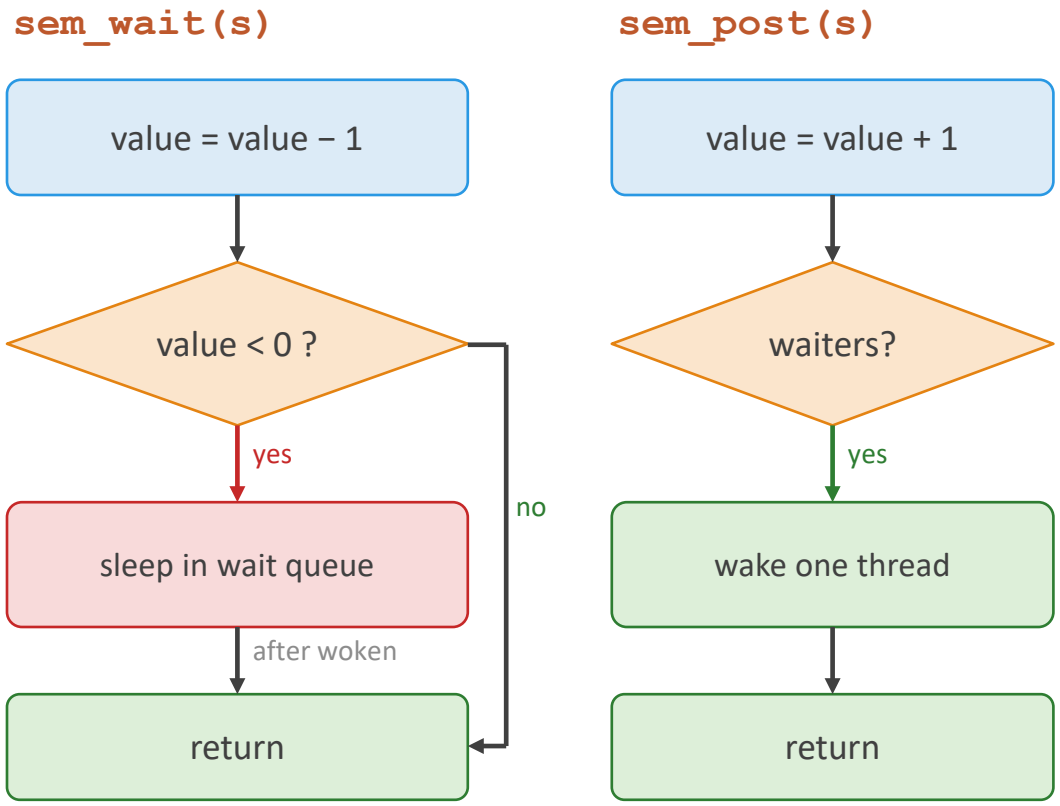
`sem_wait` → may **decrement & sleep**
`sem_post` → **increment & wake** one
Both are **atomic**

Semantics of sem_wait() and sem_post()

```
int sem_wait(sem_t *s) {
  // decrement value of s by one
  // wait if value of s is negative
}

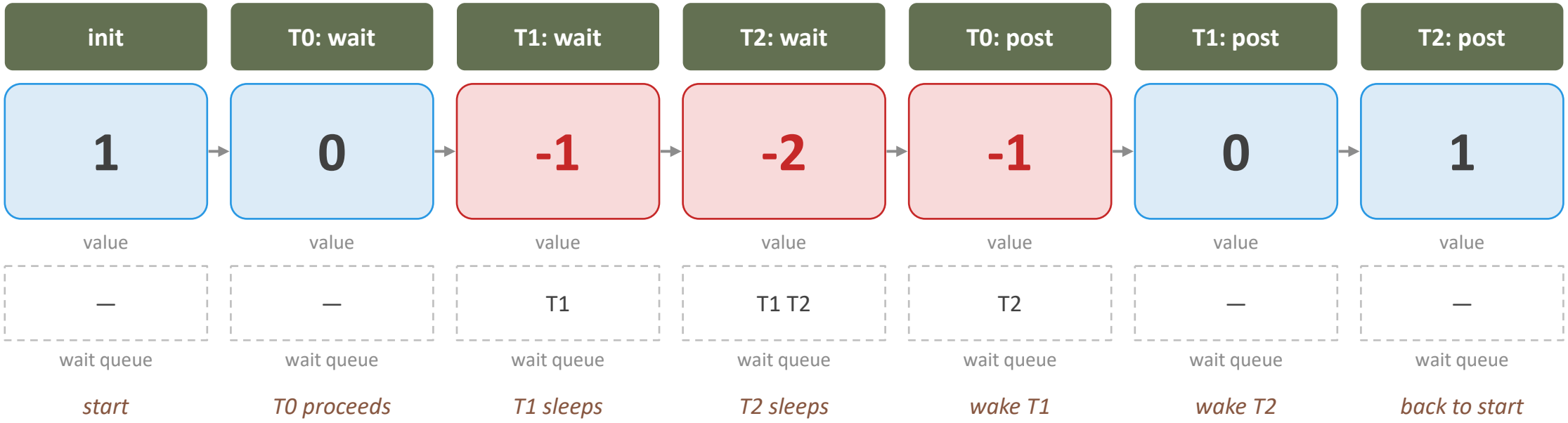
int sem_post(sem_t *s) {
  // increment value of s by one
  // if one or more threads waiting,
  //   wake one
}
```

Assume (for now) each routine executes **atomically**.
`sem_wait` may block; `sem_post` **never** blocks.



Simulation: The Value Counts the Waiters

- Invariant (Dijkstra): when **negative**, |value| = **number of waiting threads**
- Example: semaphore initialized to **1**, three threads T0, T1, T2

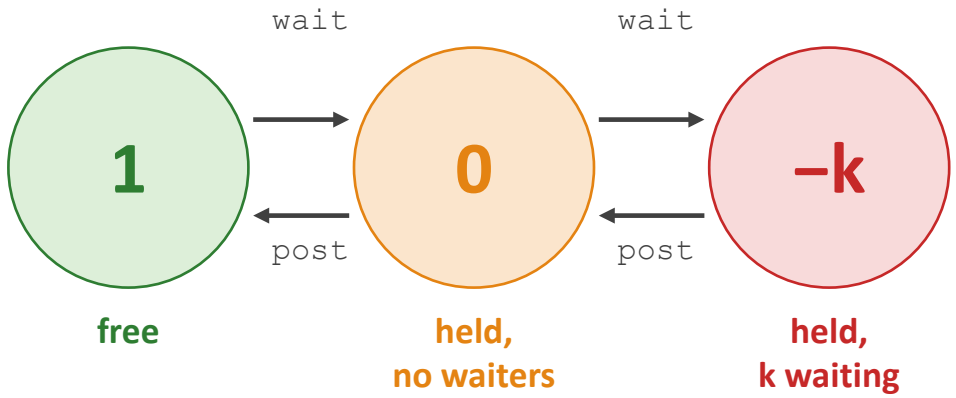


Binary Semaphores: A Semaphore as a Lock

```
sem_t m;
sem_init(&m, 0, X); // init to X; what should X be?

sem_wait(&m);
// critical section here
sem_post(&m);
```

Answer: **X = 1** → the first waiter gets through, others block

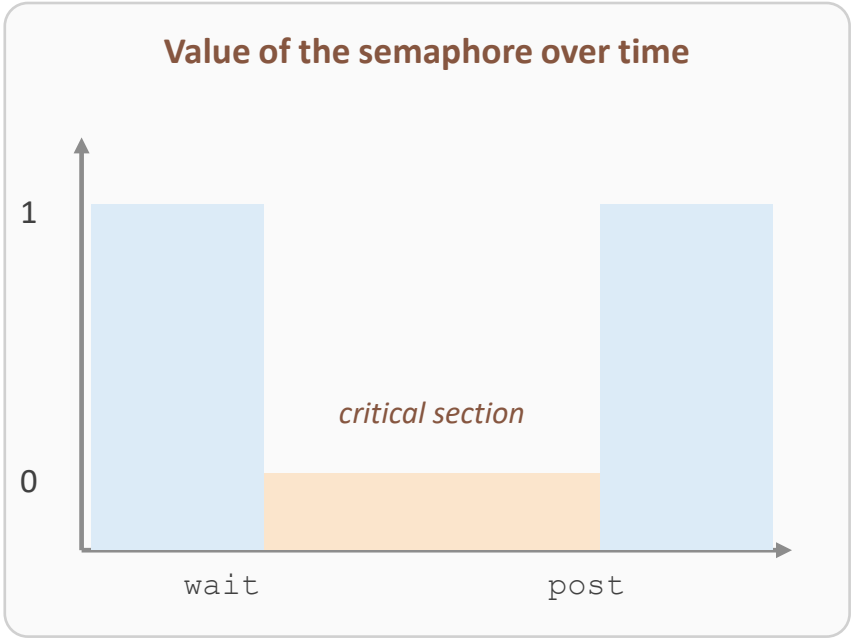


Value of `m` when used as a lock

- Locks have two states (held / not held) → **binary semaphore**
- Could be implemented more simply than a general semaphore

Trace 1: A Single Thread Uses the Lock

Value	Thread 0	Thread 1
1		
1	`call sem_wait()`	
0	`sem_wait() returns`	
0	`(crit sect)`	
0	`call sem_post()`	
1	`sem_post() returns`	



- No contention → **nobody sleeps**; value toggles 1 → 0 → 1

Trace 2: Two Threads Contend for the Lock

Val	Thread 0	State	Thread 1	State
1		Run		Ready
1	`call sem_wait()`	Run		Ready
0	`sem_wait() returns`	Run		Ready
0	`(crit sect begin)`	Run		Ready
0	*Interrupt; Switch→T1*	Ready		Run
0		Ready	`call sem_wait()`	Run
-1		Ready	*decr sem*	Run
-1		Ready	`(sem<0)→sleep`	Sleep
-1		Run	*Switch→T0*	Sleep
-1	`(crit sect end)`	Run		Sleep

Val	Thread 0	State	Thread 1	State
-1	`call sem_post()`	Run		Sleep
0	*incr sem*	Run		Sleep
0	`wake(T1)`	Run		Ready
0	`sem_post() returns`	Run		Ready
0	*Interrupt; Switch→T1*	Ready		Run
0		Ready	`sem_wait() returns`	Run
0		Ready	`(crit sect)`	Run
0		Ready	`call sem_post()`	Run
1		Ready	`sem_post() returns`	Run

Run on CPU **Ready** runnable **Sleep** blocked

Highlighted: T1 **sleeps** when value hits −1; T0's post **wakes** T1 (−1 → 0). Value never exceeds 1.

Ordering: A Parent Waits for Its Child

```c
sem_t s;

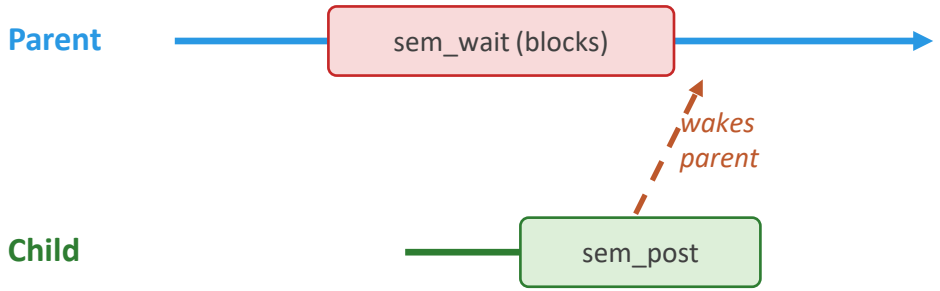
void *child(void *arg) {
    printf("child\n");
    sem_post(&s); // signal: child is done
    return NULL;
}

int main(int argc, char *argv[]) {
    sem_init(&s, 0, X); // what should X be?
    printf("parent: begin\n");
    pthread_t c;
    Pthread_create(&c, NULL, child, NULL);
    sem_wait(&s); // wait here for child
    printf("parent: end\n");
    return 0;
}
```

Desired output

```
parent: begin
child
parent: end
```

Answer: **X = 0** — nothing to give away yet



Tracing Both Cases of Parent/Child Ordering

Case 1: parent waits before child runs

Val	Parent	State	Child	State
0	`create(Child)`	Run	`(exists, can run)`	Ready
0	`call sem_wait()`	Run		Ready
-1	*decr sem*	Run		Ready
-1	`(sem<0)→sleep`	Sleep		Ready
-1	*Switch→Child*	Sleep	*child runs*	Run
-1		Sleep	`call sem_post()`	Run
0		Sleep	*inc sem*	Run
0		Ready	`wake(Parent)`	Run
0		Ready	`sem_post() returns`	Run
0		Ready	*Interrupt→Parent*	Ready
0	`sem_wait() returns`	Run		Ready

Case 2: child finishes before parent waits

Val	Parent	State	Child	State
0	`create(Child)`	Run	`(exists, can run)`	Ready
0	*Interrupt→Child*	Ready	*child runs*	Run
0		Ready	`call sem_post()`	Run
1		Ready	*inc sem*	Run
1		Ready	`wake(nobody)`	Run
1		Ready	`sem_post() returns`	Run
1	*parent runs*	Run	*Interrupt→Parent*	Ready
1	`call sem_wait()`	Run		Ready
0	*decrement sem*	Run		Ready
0	`(sem≥0)→awake`	Run		Ready
0	`sem_wait() returns`	Run		Ready

Tip: Choosing the Initial Value

Initial value = **number of resources you are willing to give away immediately** after initialization

Lock

1

one thread may enter the critical section now

Ordering

0

nothing to give away until the event happens

Buffer: empty

MAX

all MAX slots free for producers

Buffer: full

0

no data for consumers yet

Throttling

N

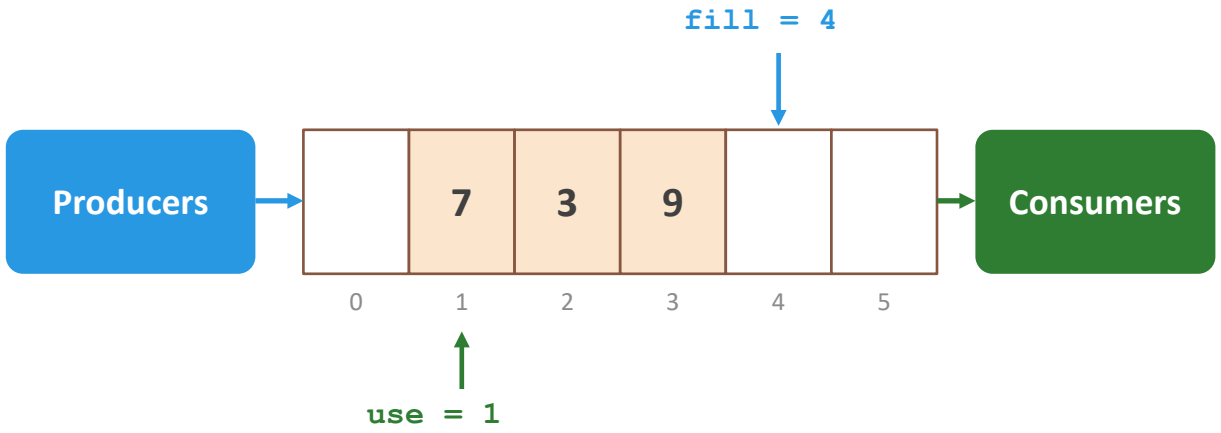
admit up to N threads at once

The Producer/Consumer (Bounded Buffer)

```
int buffer[MAX];
int fill = 0;
int use  = 0;

void put(int value) {
  buffer[fill] = value;     // F1
  fill = (fill + 1) % MAX; // F2
}

int get() {
  int tmp = buffer[use];    // G1
  use = (use + 1) % MAX;    // G2
  return tmp;
}
```



circular buffer, MAX = 6 (indices wrap with % MAX)

- Producer must **wait when full**; consumer must **wait when empty**
- `put()` / `get()` touch shared state → **critical sections**
- e.g., web-server request queue, `grep foo | wc -l`

First Attempt: Two Semaphores, empty and full

```
sem_t empty;
sem_t full;

void *producer(void *arg) {
  for (int i = 0; i < loops; i++) {
    sem_wait(&empty);    // P1
    put(i);              // P2
    sem_post(&full);     // P3
  }
}
void *consumer(void *arg) {
  int tmp = 0;
  while (tmp != -1) {
    sem_wait(&full);     // C1
    tmp = get();         // C2
    sem_post(&empty);    // C3
    printf("%d\n", tmp);
  }
}
// in main():
sem_init(&empty, 0, MAX); // MAX are empty
sem_init(&full, 0, 0);    // 0 are full
```

- **empty** = number of free slots
  - init **MAX**
- **full** = number of filled slots
  - init **0**
- Producer: wait(empty) → put → post(full)
- Consumer: wait(full) → get → post(empty)
- Each side **waits on one** semaphore and **posts the other**

Simulation: First Attempt with MAX = 1

Step	Thread / line	Action	empty	full	Buffer	Effect
0	—	*initialize*	1	0	[ ]	
1	Consumer C1	`sem_wait(&full)`	1	-1	[ ]	consumer sleeps
2	Producer P1	`sem_wait(&empty)`	0	-1	[ ]	continues
3	Producer P2	`put(0)`	0	-1	[0]	
4	Producer P3	`sem_post(&full)`	0	0	[0]	wakes consumer
5	Producer P1	`sem_wait(&empty)`	-1	0	[0]	producer sleeps (full)
6	Consumer C2	`tmp = get()`	-1	0	[ ]	reads 0
7	Consumer C3	`sem_post(&empty)`	0	0	[ ]	wakes producer
8	Producer P2	`put(1)`	0	0	[1]	cycle continues

Producer never overwrites unread data; consumer never reads an empty slot ✓

Problem: A Race When MAX > 1

- MAX = 10, **multiple** producers & consumers
- Both producers pass `sem_wait(&empty)` — many slots are free
- Then both run `put()` **concurrently**
- F1 and F2 are not atomic → **data overwritten / lost**
- Missing ingredient: **mutual exclusion** around put()/get()

Interleaving of two producers (fill starts at 0)

Pa	`F1: buffer[0] = A`
Pa	interrupted before F2
Pb	`F1: buffer[0] = B`
Pb	`F2: fill = 1`
Pa	`F2: fill = 2`

Buffer after:



A was **overwritten**; slot 1 holds **garbage**; fill = 2

Adding Mutual Exclusion (Incorrectly)

```
void *producer(void *arg) {
  for (i = 0; i < loops; i++) {
    sem_wait(&mutex);  // P0 (NEW)
    sem_wait(&empty);  // P1
    put(i);            // P2
    sem_post(&full);   // P3
    sem_post(&mutex);  // P4 (NEW)
  }
}
```

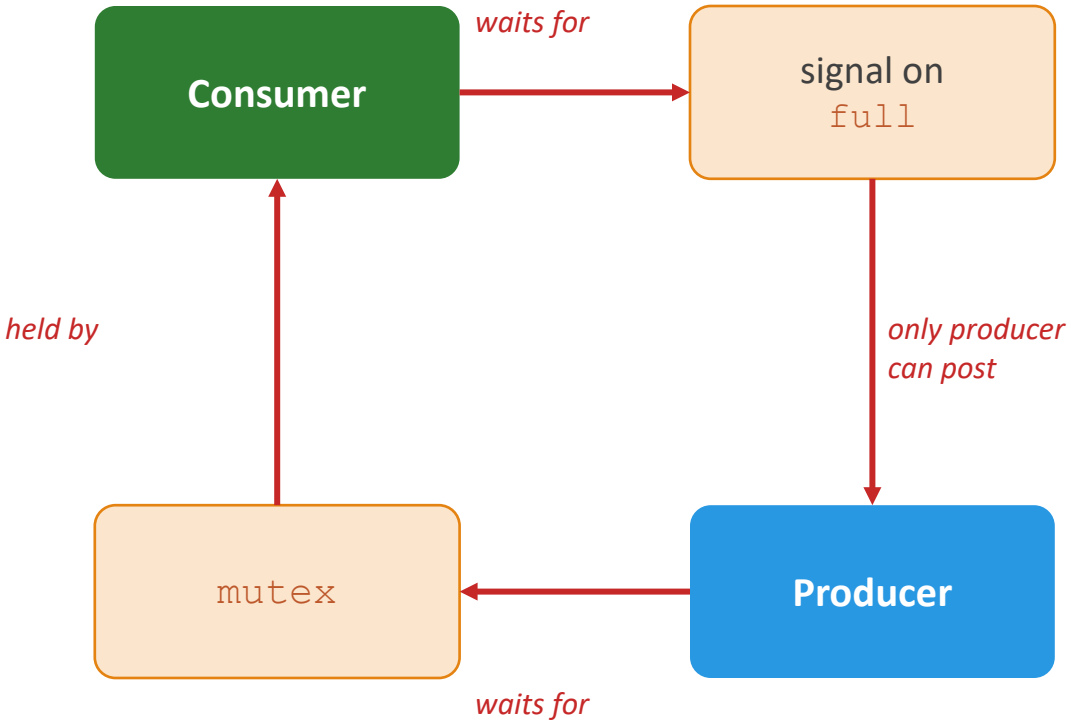
```
void *consumer(void *arg) {
  for (i = 0; i < loops; i++) {
    sem_wait(&mutex);  // C0 (NEW)
    sem_wait(&full);   // C1
    int tmp = get();   // C2
    sem_post(&empty);  // C3
    sem_post(&mutex);  // C4 (NEW)
    printf("%d\n", tmp);
  }
}
```

`mutex` initialized to **1** (binary semaphore)

The lock is acquired **before** waiting on full/empty.
What can go wrong?

Why It Deadlocks: A Cycle of Waiting

1. Buffer empty; **consumer** runs first
2. C0: acquires `mutex` (1 → 0)
3. C1: `sem_wait(&full)` → full = −1 → **sleeps holding mutex**
4. **Producer** runs; P0: `sem_wait(&mutex)` → mutex = −1 → **sleeps**
5. **Neither can proceed → deadlock**



Circular wait: each thread waits for the other

A Working Solution: Shrink the Lock's Scope

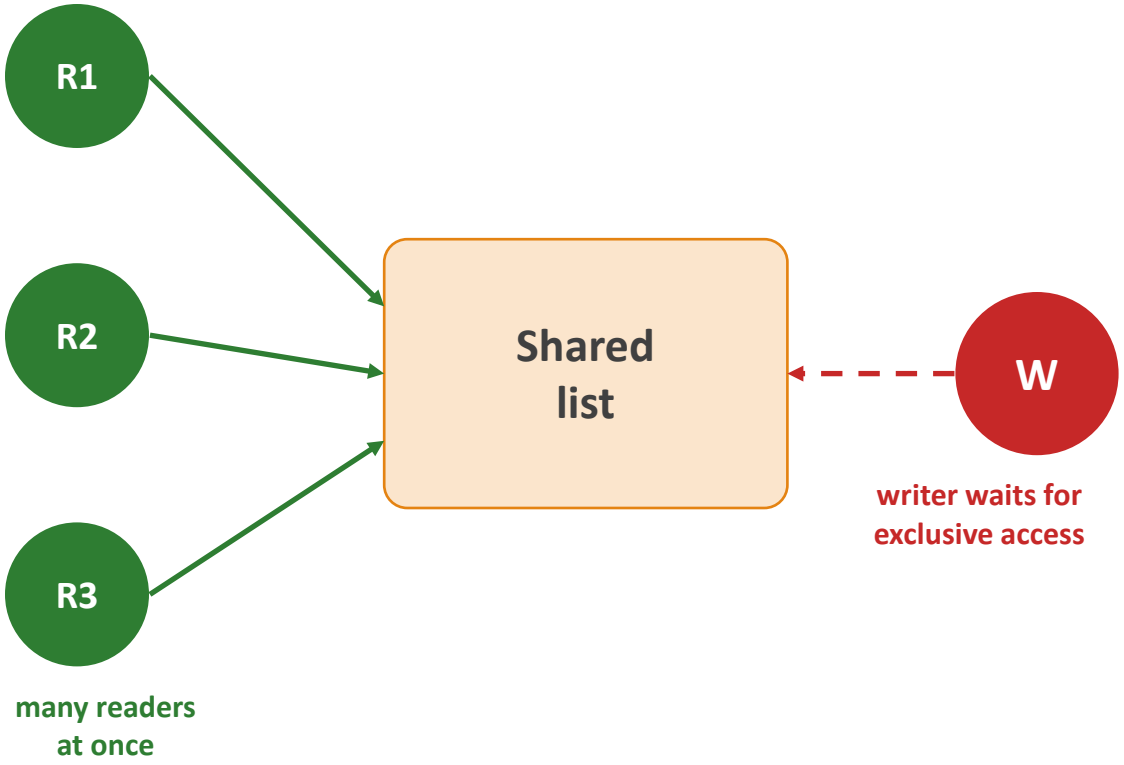
```
void *producer(void *arg) {
  for (i = 0; i < loops; i++) {
    sem_wait(&empty);   // P1
    sem_wait(&mutex);   // P1.5 lock
    put(i);             // P2
    sem_post(&mutex);   // P2.5 unlock
    sem_post(&full);    // P3
  }
}
```

```
void *consumer(void *arg) {
  for (i = 0; i < loops; i++) {
    sem_wait(&full);    // C1
    sem_wait(&mutex);   // C1.5 lock
    int tmp = get();    // C2
    sem_post(&mutex);   // C2.5 unlock
    sem_post(&empty);   // C3
    printf("%d\n", tmp);
  }
}
```

- Wait on **full/empty outside** the lock; hold `mutex` only around **put()/get()** → no thread sleeps while holding the lock

Reader-Writer Locks: Motivation

- Shared list: **inserts** modify, **lookups** only read
- Many lookups can safely run **concurrently**
- An insert needs **exclusive** access
- → **Reader-writer lock** [Courtois et al., 1971]



Holder \ Requester	Reader	Writer
Reader(s)	✓ **allowed**	✗ **wait**
Writer	✗ **wait**	✗ **wait**

A Simple Reader-Writer Lock with Semaphores

```
typedef struct _rwlock_t {
  sem_t lock;      // binary sem (basic lock)
  sem_t writelock; // ONE writer / MANY readers
  int   readers;   // #readers in crit. sect.
} rwlock_t;

void rwlock_init(rwlock_t *rw) {
  rw->readers = 0;
  sem_init(&rw->lock, 0, 1);
  sem_init(&rw->writelock, 0, 1);
}

void rwlock_acquire_readlock(rwlock_t *rw) {
  sem_wait(&rw->lock);
  rw->readers++;
  if (rw->readers == 1) // first reader
    sem_wait(&rw->writelock);
  sem_post(&rw->lock);
}
```

```
void rwlock_release_readlock(rwlock_t *rw) {
  sem_wait(&rw->lock);
  rw->readers--;
  if (rw->readers == 0) // last reader
    sem_post(&rw->writelock);
  sem_post(&rw->lock);
}

void rwlock_acquire_writelock(rwlock_t *rw) {
  sem_wait(&rw->writelock);
}
void rwlock_release_writelock(rwlock_t *rw) {
  sem_post(&rw->writelock);
}
```

First reader grabs `writelock` for all readers
Last reader releases it → a writer may enter

Simulation: Readers Can Starve a Writer

Step	Event	readers	writelock	Inside	Waiting
1	R1 acquires read lock (first reader)	**1**	**0**	R1	—
2	R2 acquires read lock	**2**	**0**	R1, R2	—
3	W calls acquire_writelock	**2**	**-1**	R1, R2	W
4	R1 releases (not last)	**1**	**-1**	R2	W
5	R3 arrives → enters at once	**2**	**-1**	R2, R3	W
…	readers keep arriving …	**≥1**	**-1**	…	W starves

- Works, but **not fair**: readers can **starve** writers
  - Fix idea: stop new readers once a writer is waiting

Tip: Hill's Law

Simple and dumb can be better.

RW locks add **overhead** and often don't beat a simple, fast lock.

Hill: direct-mapped caches often beat fancy set-associative ones.

"Big and dumb is better."

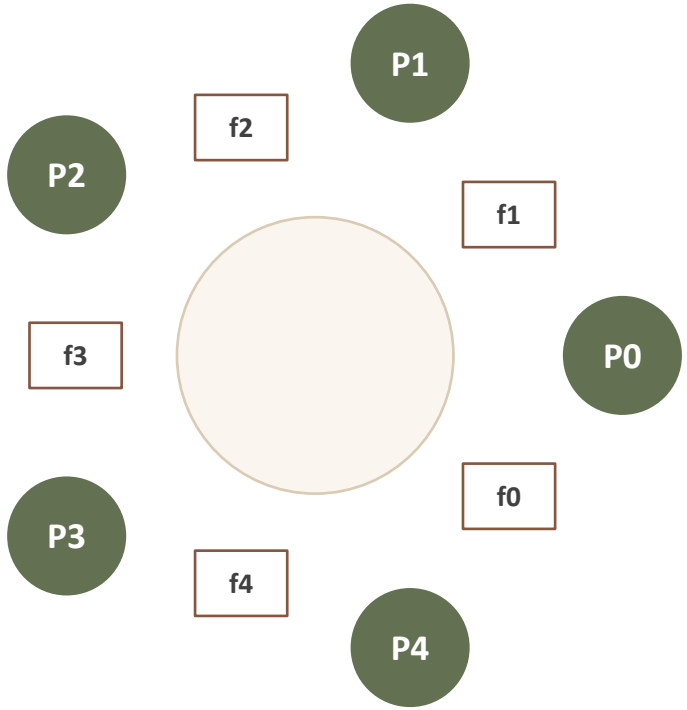
The Dining Philosophers Problem

Each philosopher p (0..4):

```
while (1) {
  think();
  get_forks(p);
  eat();
  put_forks(p);
}
```

```
int left(int p)  { return p; }
int right(int p) { return (p + 1) % 5; }
```

- Goals: **no deadlock**, **no starvation**, **high concurrency**
- One semaphore per fork: `sem_t forks[5]`, each init **1**



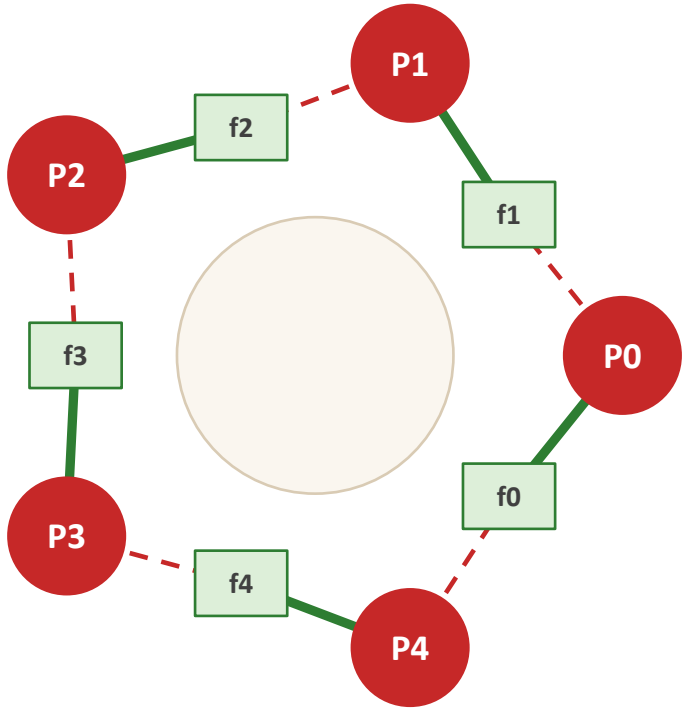
P0 uses f0 (left) and f1 (right); P4 uses f4 and f0

Broken Solution: Everyone Grabs the Left Fork

```
void get_forks(int p) {
  sem_wait(&forks[left(p)]);
  sem_wait(&forks[right(p)]);
}

void put_forks(int p) {
  sem_post(&forks[left(p)]);
  sem_post(&forks[right(p)]);
}
```

- All grab **left** fork first, then try **right**
- **Each holds one fork, waits for its neighbour's → deadlock**

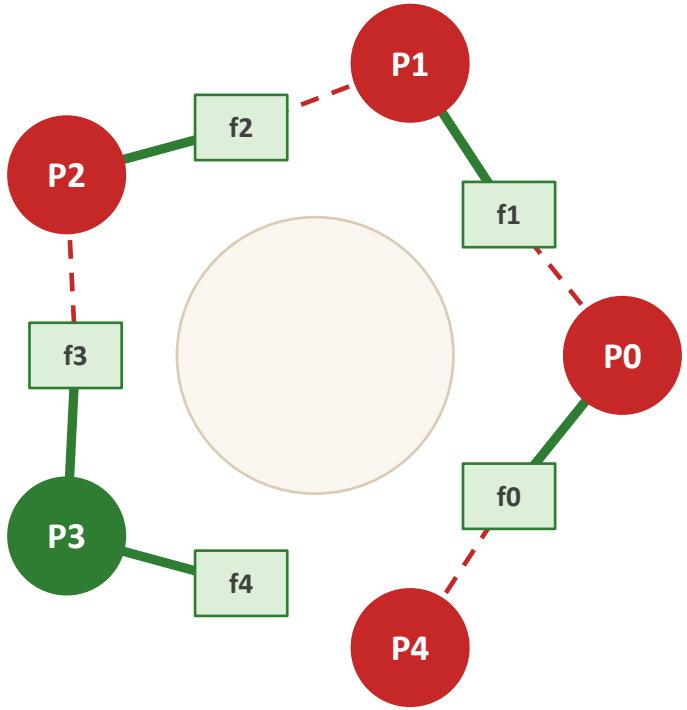


green = held, red dashed = waiting

Solution: Break the Dependency

```
void get_forks(int p) {
  if (p == 4) {
    sem_wait(&forks[right(p)]);
    sem_wait(&forks[left(p)]);
  } else {
    sem_wait(&forks[left(p)]);
    sem_wait(&forks[right(p)]);
  }
}
```

- P4 asks for **f0 first** → blocks holding nothing
- f4 stays free → **P3 can eat**; cycle is broken ✓
- Also: cigarette smokers, sleeping barber

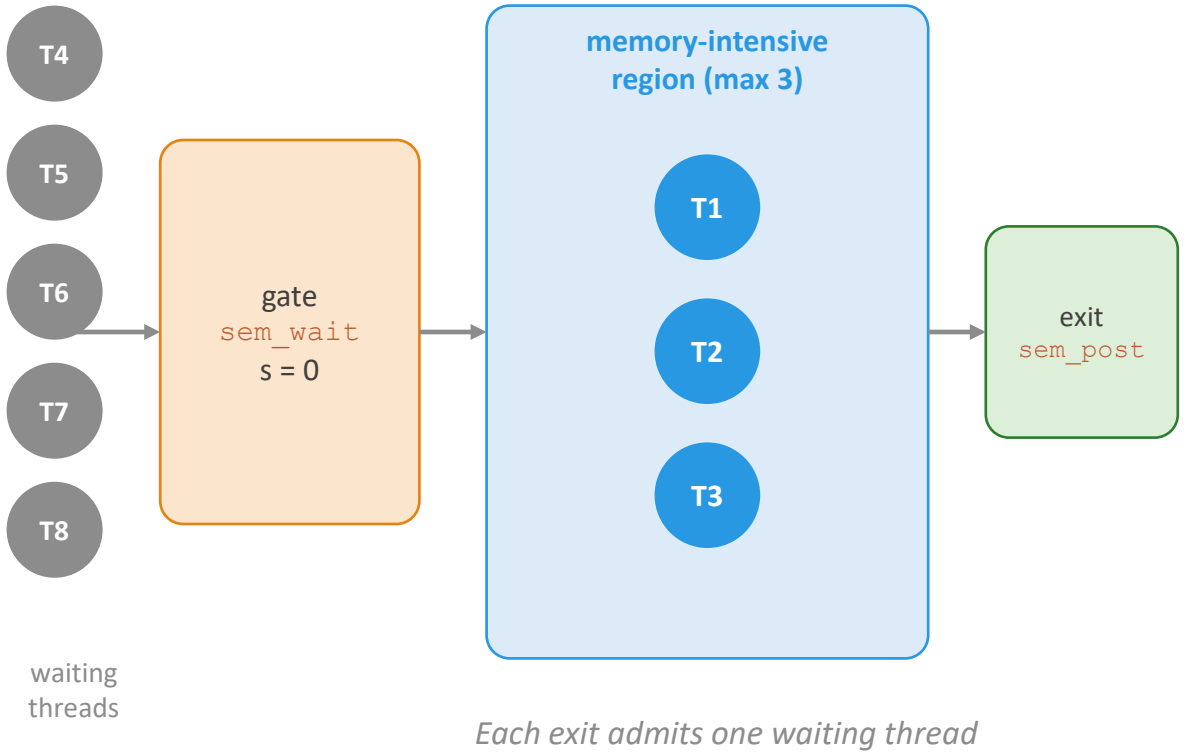


P3 holds f3 + f4 and eats; P4 waits holding nothing

Thread Throttling: Admission Control

```
sem_init(&s, 0, N);    // e.g. N = 3
sem_wait(&s);
memory_intensive_region();
sem_post(&s);
```

- Hundreds of threads; one **memory-intensive** region
- All inside at once → exceeds RAM → **thrashing**
- Semaphore initialized to **N** caps concurrency at N



Implementing Semaphores: Zemaphores

```
typedef struct __Zem_t {
  int value;
  pthread_cond_t  cond;
  pthread_mutex_t lock;
} Zem_t;

// only one thread can call this
void Zem_init(Zem_t *s, int value) {
  s->value = value;
  Cond_init(&s->cond);
  Mutex_init(&s->lock);
}
```

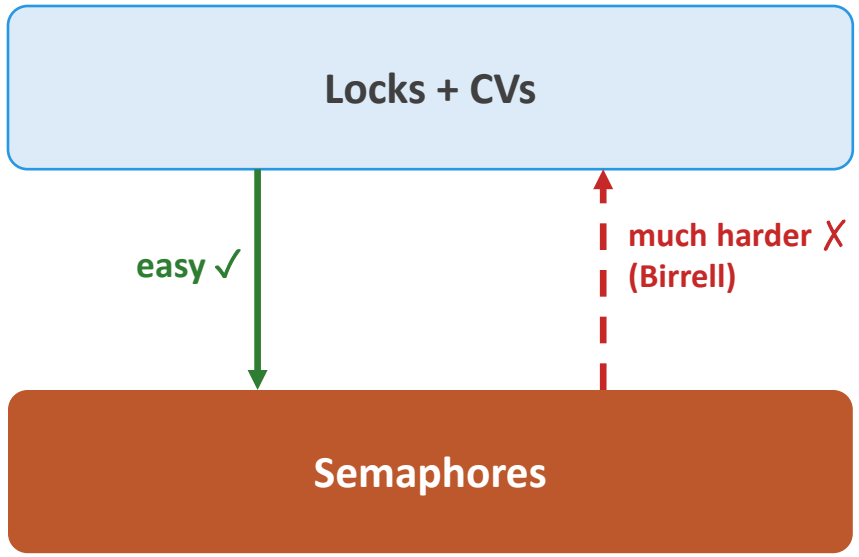
1 lock + 1 CV + 1 integer
Lock → atomicity of wait/post
CV → sleep & wake; `while` rechecks (Mesa)

```
void Zem_wait(Zem_t *s) {
  Mutex_lock(&s->lock);
  while (s->value <= 0)
    Cond_wait(&s->cond, &s->lock);
  s->value--;
  Mutex_unlock(&s->lock);
}

void Zem_post(Zem_t *s) {
  Mutex_lock(&s->lock);
  s->value++;
  Cond_signal(&s->cond);
  Mutex_unlock(&s->lock);
}
```

Zemaphores vs. Dijkstra's Semaphores

	Dijkstra semaphore	Zemaphore
Value can be negative?	Yes	No (≥ 0)
Negative value means	# of waiters	—
Built from	primitive	lock + CV
Matches	original definition	Linux today



Tip: Be careful with generalization. "Don't generalize; generalizations are generally wrong." — Lampson
Semaphores generalize locks and CVs, but since CVs are hard to build from them, the generalization is less general than it looks.

Summary

Lock (binary) — init 1

wait/post around the critical section

Ordering — init 0

waiter waits; signaller posts when done

Bounded buffer — MAX / 0 + mutex

lock only around put/get to avoid deadlock

Reader-writer — 1, 1

first reader locks writers out; beware starvation

Dining philosophers — forks = 1

one philosopher reverses order → no cycle

Throttling — init N

at most N threads in a region

Homework (Downey's Little Book of Semaphores): fork/join · rendezvous · barrier · reader-writer (+ no-starve) · no-starve mutex

Thank You!!